

17.5

ConfigMaps & Secrets

Separating config from code in Kubernetes

Java Backend Architecture Interview Series • Tutorial Edition • Easy Diagrams & Examples

Why Separate Config from Code?

A 12-factor application stores all configuration in environment variables, never baked into the Docker image. This lets you use the SAME image in dev, staging, and prod with different config values. Kubernetes provides ConfigMaps for non-sensitive config and Secrets for sensitive data like passwords and API keys.

■ **Simple Analogy:** Your app is a recipe (code). ConfigMap is the ingredient list (config). Secret is the locked safe with the secret recipe (passwords). The same recipe can be cooked with different ingredient lists (environments).

ConfigMap — Non-Sensitive Configuration

```
# Create ConfigMap from YAML
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  DB_HOST: postgres-service
  DB_PORT: "5432"
  LOG_LEVEL: INFO
  MAX_CONNECTIONS: "50"

# Inject into Pod as environment variables
spec:
  containers:
    - name: java-app
  envFrom:
    - configMapRef:
        name: app-config # ALL keys become env vars
```

Secret — Sensitive Data

Secrets are base64-encoded (NOT encrypted by default!). Always enable etcd encryption at rest in production. For production, use External Secrets Operator to pull from AWS Secrets Manager or Vault.

```
# Create Secret (values must be base64-encoded)
apiVersion: v1
kind: Secret
metadata:
  name: db-credentials
  type: Opaque
data:
  DB_PASSWORD: c2VjcmV0MTIz # base64 of 'secret123'
  API_KEY: bXktYXBpLWtleQ== # base64 of 'my-api-key'

# Inject into Pod
env:
  - name: DB_PASSWORD
    valueFrom:
      secretKeyRef:
        name: db-credentials
        key: DB_PASSWORD
```

Three Ways to Inject Config into Pods

Method	How	Best For	Live Reload?
envFrom (all keys)	All ConfigMap keys as env vars	Simple config	No (pod restart needed)
env.valueFrom	One specific key as named env var	Selective inject	No
Volume Mount	ConfigMap mounted as files	Config files, app.properties	No (~60s delay)

Production Secret Management

Warning: Base64 is NOT encryption. Anyone with kubectl access can decode it.

Production options:

1. **External Secrets Operator** — syncs secrets from AWS Secrets Manager / GCP Secret Manager
2. **Sealed Secrets (Bitnami)** — encrypt secrets for git storage
3. **Vault Agent Injector** — HashiCorp Vault injects secrets as sidecar

■ **Real-World Example:** Airbnb uses External Secrets Operator to sync all secrets from AWS Secrets Manager into Kubernetes Secrets at startup. No secrets are stored in Git. Rotation is automatic — Secrets Manager rotates the value, ESO syncs to K8s, pod restarts pick up the new value.